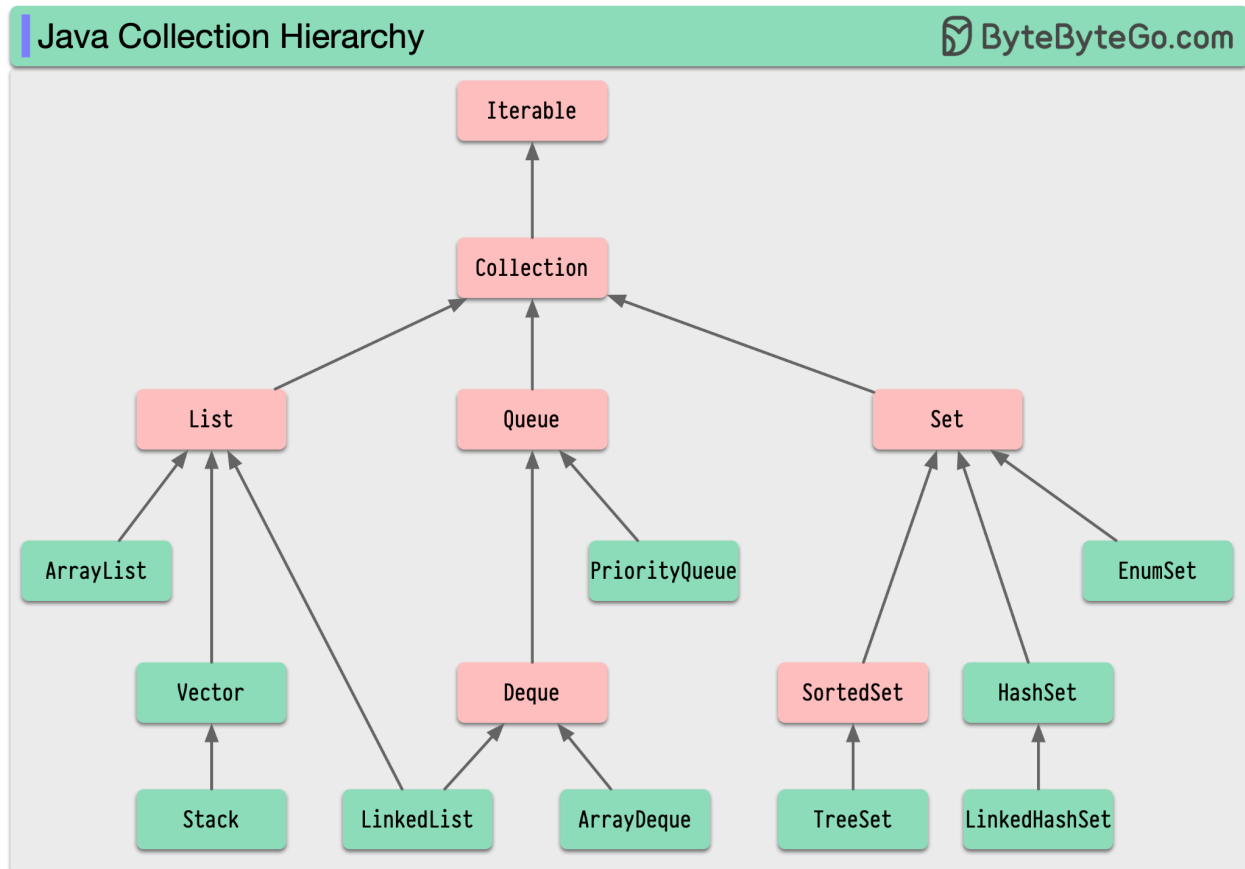


Java Collections Framework Notes



Introduction and Overview

- **Purpose of the Video:** This tutorial is designed to teach the Java Collections Framework specifically for Data Structures and Algorithms (DSA) interviews and day-to-day software engineering work.
- **What is a Collection?** A collection is a framework that allows a group of objects to be treated as a single unit. It provides standard utility classes for managing various kinds of collections and resides in the `java.util` package.

Core Interfaces and Hierarchy

- **Core Interfaces:**
 - **Iterable Interface:** Sits at the absolute top of the chain (part of `java.lang`).
 - **Collection Interface:** Extends the `Iterable` interface. It is an interface, not a concrete class, meaning it cannot be instantiated directly.
- **Sub-Interfaces of Collection:**

- List
- Queue (which is extended by Deque)
- Set (extended by SortedSet, which is further extended by NavigableSet)
- **The Map Interface:** Notably, the Map interface does not extend the Collection interface; it exists independently. It is extended by SortedMap and NavigableMap.
- **Concrete Implementations:**
 - **List:** Implemented by Vector, ArrayList, and LinkedList.
 - **Queue / Deque:** LinkedList implements Deque, PriorityQueue implements Queue, ArrayDeque implements Deque.
 - **Set:** Implemented by HashSet and TreeSet.
 - **Map:** Implemented by HashMap, TreeMap, and LinkedHashMap.

The Collection and List Interfaces

- **Why Collection extends Iterable:** Because the Collection interface extends Iterable, every single collection (Lists, Sets, Queues) in Java inherently supports the for-each loop.
- **Collection Methods:** Any class implementing Collection will have standard methods like contains(), add(), remove(), and clear().
- **The List Interface:**
 - Maintains elements in the exact order they are inserted.
 - Allows duplicate elements.
 - Supports position-based access (zero-based indexing).
- **ArrayList:**
 - A dynamically resizable array. Used when the exact size of the array is unknown ahead of time.
 - **Internal Working:** It uses a normal array of a default capacity. When full, it automatically creates a new, larger array (typically 1.5x the size), copies the old elements over, and garbage collects the old array.
- **Vector:** Very similar to ArrayList but is "thread-safe." It synchronizes concurrent calls, which introduces a slight performance penalty. ArrayList is almost always preferred in standard DSA problems.
- **LinkedList:** Uses a doubly-linked list under the hood. It implements both List and Deque.

ArrayList vs LinkedList Comparison

Feature	ArrayList	LinkedList
Access Time	Offers fast $O(1)$ random access by index.	Slow $O(N)$ random access.
Insertions/Deletions	Insertions/deletions in the middle take $O(N)$ time because elements must be	Allows for fast $O(1)$ insertions/deletions once the position is found.

Feature	ArrayList	LinkedList
	shifted.	
Takeaway	Default to ArrayList 99% of the time when you need a List.	Only use LinkedList for very specific Queue/Deque operations.

ArrayList Deep Dive & Methods

- **Declaration:** Always code against the interface:

```
List<Integer> list = new ArrayList<>();
```

- **Wrapper Classes:** Collections require reference types (e.g., Integer, Float) instead of primitive types (int, float). Java performs automatic boxing and unboxing.
- **Common Methods:**
 - `add(element)`: Appends the element to the end of the list.
 - `set(index, element)`: Replaces the element at the specified index and returns the old element.
 - `indexOf(element) / lastIndexOf(element)`: Returns the first/last index of the given element.
 - `addAll(collection)`: Adds all elements from another collection into the list.
- **Copying an ArrayList:** You can pass a collection to the constructor to create a copy:

```
new ArrayList<>(existingList);
```

- **Sublists:** `subList(fromIndex, toIndex)` returns a slice from `fromIndex` (inclusive) to `toIndex` (exclusive). Important: It returns a shallow copy/view. Modifying the sublist modifies the original underlying list.

ListIterator and LinkedList

- ListIterator is an interface that extends Iterator and allows bi-directional traversal.
- Includes methods like `hasPrevious()` and `previous()` in addition to `hasNext()` and `next()`.
- Useful for traversing a LinkedList (which uses a doubly-linked list internally).
- `previous()` internally decrements the index first and then returns the element.

Converting a List to an Array

- Use the `toArray()` method when you need to return a normal array (e.g., required by some problem signatures).
- Convention: Pass an array of size 0 (e.g., `list.toArray(new Integer[0])`). Java will

automatically create a new array of the exact size needed.

The Queue Interface

- Queue extends the Collection interface.
- **Adding Elements:**
 - add(element): Throws an IllegalStateException if the queue is full.
 - offer(element): Returns false if the queue is full. (Preferred to avoid exceptions)
- **Removing Elements:**
 - remove(): Throws a NoSuchElementException if the queue is empty.
 - poll(): Returns null if the queue is empty. (Preferred)
- **Examining Elements:**
 - element(): Throws an exception if the queue is empty.
 - peek(): Returns null if the queue is empty. (Preferred)

Deque and Stack

- **Deque Interface:** Extends the Queue interface to allow double-ended operations. Elements can be inserted and removed from both ends. It can be used as a FIFO Queue or a LIFO Stack.
- **Implementations:** ArrayDeque and LinkedList both implement the Deque interface.
- **Usage Recommendations:**
 - Normal Queue (FIFO): Instantiate using LinkedList.
 - Stack (LIFO): Instantiate using the Stack class or ArrayDeque.
 - Double-Ended Queue: Instantiate using ArrayDeque.

Priority Queue

- PriorityQueue implements the Queue interface.
- Elements are ordered based on priority, not insertion order. Priority is determined by natural ordering or a custom Comparator.
- Elements are not sorted when traversing via an iterator.

Comparable vs Comparator

Feature	Comparable	Comparator
Interface Definition	Defines "natural ordering" for a class.	Defines a "total ordering", overriding natural sorting.

Feature	Comparable	Comparator
Method Overridden	<code>compareTo(Object o)</code> (return negative if current is less, positive if greater, 0 if equal).	<code>compare(Object o1, Object o2)</code>
Location	Implemented directly within the custom class.	Passed as a separate object/lambda (e.g., to <code>PriorityQueue</code> constructor). Lambdas can be used for concise comparator implementations (e.g., <code>(a, b) -> b - a</code> for descending order).

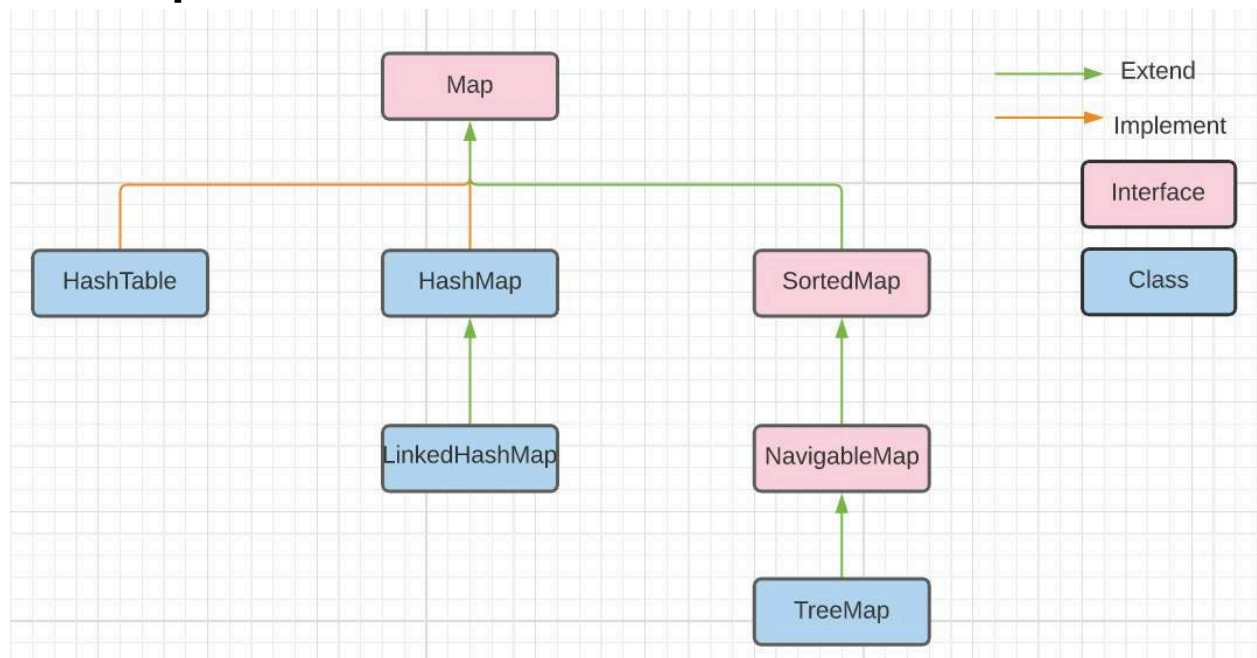
Set Interface and Implementations

- **Set Interface:** A collection that contains unique elements (no duplicates allowed).
- **HashSet:** Implementation of Set using a hash table. Provides fast $O(1)$ average time complexity for insertions and deletions. Elements are completely unordered.
- **LinkedHashSet:** Extends HashSet and maintains a doubly-linked list across its elements to preserve insertion order. It has a slight performance overhead compared to HashSet.
- **Core Set Operations:**
 - `retainAll(Collection c)`: Performs set intersection.
 - `removeAll(Collection c)`: Performs set difference.
 - `addAll(Collection c)`: Performs set union.
- **Custom Objects in Sets:** When storing custom class objects in a HashSet (or as keys in a HashMap), you must override both the `hashCode()` and `equals()` methods. If not, the default Object implementations (which rely on memory addresses) will be used, preventing the collection from correctly identifying logical duplicates or successfully using methods like `contains()`.

SortedSet, NavigableSet, and TreeSet

- **SortedSet:** Extends the Set interface to provide sorting functionality. Elements are sorted according to natural ordering or a provided comparator. Contains methods like `first()` and `last()`.
- **NavigableSet:** Extends SortedSet with navigation methods to find closest matches. Key methods include `pollFirst()`, `pollLast()`, `ceiling()`, `floor()`, `higher()`, and `lower()`. Useful for range interval queries.
- **TreeSet:** A concrete implementation of NavigableSet backed by a balanced binary search tree. Operations like add, remove, and search take $O(\log N)$ time. It maintains elements in a sorted order.

The Map Interface



- Unlike List, Queue, and Set, the **Map interface does not extend the Collection interface**. It represents a mapping between keys and values (an entry).
- Maps do not allow duplicate keys; each key maps to exactly one value. Values, however, can be duplicated.
- Keys and values must be objects (reference types), not primitives.
- **Map Views:** A Map can be viewed as a collection in three ways:
 - `keySet()`: Returns a Set of keys.
 - `values()`: Returns a Collection of values.
 - `entrySet()`: Returns a Set of `Map.Entry<K, V>` objects. This is the primary way to iterate over a map.
- **Core Map Methods:**
 - `put(key, value)`: Inserts or updates a key-value pair. Returns the previous value associated with the key or null.
 - `get(key)`: Retrieves the value for a key, or null if the key doesn't exist.
 - `remove(key)`: Removes the entry associated with the key.
 - `containsKey(key)` / `containsValue(value)`: Checks for existence.
 - `getOrDefault(key, defaultValue)`: Returns the value if the key exists, otherwise returns the specified default value. Useful to avoid `NullPointerException`.
 - `computeIfAbsent(key, mappingFunction)`: If the key is not present, computes its value using the provided function and enters it into the map (very useful for creating adjacency lists in graphs).

Map Implementations

- **HashMap:** The most common implementation, backed by a hash table. Provides average $O(1)$ time complexity for get and put. Elements are unordered. Permits one null key. Requires keys to have correctly overridden hashCode() and equals() methods.
- **LinkedHashMap:** Extends HashMap and maintains a doubly-linked list to preserve either insertion order or access order (useful for LRU caches). Slightly slower than HashMap due to the overhead of maintaining the linked list.
- **Hashtable:** An older, thread-safe version of HashMap. It does not allow any null keys or values. Generally, HashMap is preferred unless thread safety is strictly required.

SortedMap, NavigableMap, and TreeMap

- **SortedMap:** Extends Map to maintain keys in sorted order. Methods include firstKey() and lastKey().
- **NavigableMap:** Extends SortedMap with navigation methods like ceilingEntry(), floorEntry(), higherEntry(), and lowerEntry().
- **TreeMap:** Implementation of NavigableMap backed by a balanced binary search tree. Keys are sorted either by natural ordering or by a specified Comparator. Provides $O(\log N)$ complexity for basic operations. Useful when key ordering is required.

Utility Classes: Arrays and Collections

- **Sorting:** Use Arrays.sort(array) for normal arrays and Collections.sort(list) for lists. Both can take a custom Comparator. Collections.reverseOrder() provides a comparator for descending natural order.
- **Conversion:** Arrays.asList(elements...) converts an array (of reference types) or a variable number of arguments into a List.
- **Binary Search:** Collections.binarySearch(list, key) can be used on a sorted list to find elements, but it's typically better to implement your own for specific DSA bounds.